

计算机组成原理 第5章 指令系统 知识点整理

目录结构：5.1 指令系统概述 → 5.2 指令格式 → 5.3 寻址方式 → 5.4 指令类型 → 5.5 指令格式设计 → 5.6 CISC和RISC → 5.7 指令系统举例

5.1 指令系统概述

5.1.1 指令的层次

计算机中的指令包括：高级语言指令、汇编语言指令、机器指令、微指令。

层次	类别	说明
高级语言指令	软件	需编译或解释成多条机器指令（1:m）
汇编语言指令	软件	汇编成一条机器语言指令（1:1）
机器指令	硬件	第5章主要介绍的对象
微指令	硬件	由若干条微指令实现一条机器指令（n:1），第6章介绍

5.1.2 指令系统的定义

一台计算机中**所有指令的集合**称为该计算机的指令系统，也称指令集。

5.1.3 完善指令系统的要求

要求	说明
完备性	指令系统种类齐全、功能完备，能够编写任何可计算的程序
规整性	对称性：寄存器和存储单元都可被同等对待，所有指令都可以使用各种寻址方式。 均齐性：支持不同数据类型（字节、字、双字整数、十进制数、单/双精度浮点）
有效性	编写的程序能高效率运行，方便硬件和编译器实现，占用存储资源少

要求	说明
兼容性	新一代计算机的指令系统应兼容旧指令系统（旧程序无须修改即可在新计算机上运行）
可扩展性	操作码预留编码空间，以便扩展指令功能

5.2 指令格式

指令包括**操作码OP**和**地址码A**两部分：

- **操作码OP（Operation code）**：解决进行何种操作的问题
- **地址码A**：解决处理什么操作数的问题，可以包括多个操作数

通过何种方式获取操作数由**寻址方式**决定，寻址方式决定操作数存放的位置和访问方式。

寻址方式的编码位置：

- 可在地址码A中：如PDP-11指令集、Intel x86指令集
- 可在操作码OP中：如MIPS指令集、RISC-V指令集

5.2.1 指令字长度

指令字长度（指令长度/指令字长）：指令机器码的长度。

定长指令系统

指令长度固定，RISC计算机中多采用。

方面	说明
优点	结构简单，有利于CPU取指令、译码和指令顺序寻址，方便硬件执行
缺点	平均指令长度较长，冗余状态较多，不容易扩展

变长指令系统

指令长度可变，CISC计算机中多采用（如Intel x86）。

方面	说明
优点	结构灵活，冗余状态较少，平均指令长度较短，可扩展性好
缺点	取指令和译码不方便，取指过程可能涉及多次访存， 下一条指令地址必须在译码后才能确定，增加硬件控制难度

指令字长应为字节的整倍数。

按与机器字长的关系分类

类型	说明
半字长指令	指令字长为机器字长的一半，CPU访问一次主存可读取两条指令
单字长指令	指令字长等于机器字长，CPU访问一次主存只能读取一条指令
多字长指令	指令字长为机器字长的多倍，CPU需多个存储周期才能读取一条指令

通常将最常用的指令设计为短指令格式（半字长或单字长）。

5.2.2 指令地址码

指令中地址码A的作用随指令类型和寻址方式的不同而不同：

- 可能是一个**操作数**： `MOV AH, 20H`
- 也可能是**操作数的地址**（主存地址、寄存器编号、外部设备端口地址）：
 - `MOV AH, [2000H]`（主存地址）
 - `MOV AH, AL`（寄存器编号）
 - `IN AL, DX`（外部设备端口地址）
- 还可能用于**计算地址的偏移量**： `MOV AX, [SI + 5]`

按操作数地址数量分类

1) 三地址指令

格式： $A3 \leftarrow (A1) \text{ OP } (A2)$

示例： `add rd, rs, rt ; R[rd]=R[rs]+R[rt]`

- 若三个地址均为存储器地址（OP=6位，存储容量=16KB，地址=14位）：指令长度=6+14+14+14=48位

- 若三个地址均为寄存器地址（OP=6位，寄存器=32个，地址=5位）：指令长度=6+5+5+5=21位

2) 双地址指令

格式： $A1 \leftarrow (A1) \text{ OP } (A2)$

示例： `ADD AL, BL` ； $AL \leftarrow AL + BL$

三种类型：

类型	名称	示例	速度
RR（寄存器-寄存器）型	<code>MOV AL, BL</code>	最快，使用最多	
RS（寄存器-存储器）型	<code>MOV [2000H], AL</code>	中等	
SS（存储器-存储器）型	<code>MOV [2000H], [3000H]</code>	最慢，使用最少	

3) 单地址指令

(1) 单目运算指令

格式： $A1 \leftarrow \text{OP } (A1)$

示例： `INC AL` ； $AL \leftarrow AL + 1$

(2) 隐含操作数的双目运算指令

格式： $A1 \leftarrow (AC) \text{ OP } (A1)$

示例： `MUL BL` ；AL与BL相乘，结果送AX，隐含操作数AX

4) 零地址指令

(1) 不需要操作数

示例： `NOP` （空操作指令）

(2) 隐含操作数的单目运算指令

示例： `DAA` （BCD码调整指令，将AL中的内容进行BCD码调整，隐含AL）

```
MOV AL, 43H
MOV BL, 29H
ADD AL, BL      ; AL=6BH
DAA             ; AL=72H (43+29=72, BCD调整)
```

5.2.3 指令操作码

1) 定长操作码

操作码OP长度固定，且在指令中的位置固定。

假设指令系统有m条指令，则操作码位数n应满足： $n \geq \log_2m$

例：m=128， $\log_2m=7$ ，则 $n \geq 7$ 。

2) 变长操作码（扩展操作码技术）

操作码OP长度可变，位置不固定。采用**扩展操作码技术**实现：操作码长度随地址码数目的增加而减少。

例题5.1：假设扩展操作码指令系统中有三地址指令15条、双地址指令14条、单地址指令22条，该指令系统最多可设计多少条零地址指令？

解：

剩余状态逐级传递：

- 三地址指令剩余： $2^4 - 15 = 1$
- 双地址指令剩余： $1 \times 2^4 - 14 = 2$
- 单地址指令剩余： $2 \times 2^4 - 22 = 10$
- 零地址指令： $10 \times 2^4 = 160$

结论：最多可设计160条零地址指令。

编码示意：

指令类型	编码前缀	数量
三地址指令	0000 ~ 1110（前4位≠1111）	15条
双地址指令	1111 + 0000~1101	14条
单地址指令	1111 + 1110 + 0000~0101 及 1111 + 1111 + 0000~0101	22条
零地址指令	剩余编码空间	160条

5.3 寻址方式

指令和数据（操作数）存放在主存中，只有获得有效地址（EA，Effective Address），CPU才能访问所需的指令和数据。

寻址方式：寻找指令或操作数有效地址的方法。

分类：

- **指令寻址方式**（找下一条指令）
- **操作数寻址方式**（找操作数）

5.3.1 指令寻址方式

1) 顺序寻址方式

大部分程序中机器指令序列在主存中按顺序存放。只要知道第一条指令的有效地址，通过增加一条指令占用的主存单元数量，即可知道下一条指令的有效地址。

- **程序计数器PC (Program Counter)：**用于存放下一条指令的地址
- 主存按字节编址：单字节指令PC+1，双字节指令PC+2，以此类推
- **PC应具有自动加1的功能（指令预取）**

2) 跳跃寻址方式

程序中有分支或转移时，改变程序执行顺序。下一条指令地址由指令本身及需要测试的条件决定。

- **无条件转移指令、条件转移指令**都采用跳跃寻址方式
- 取出 `jmp 1003` 时，PC先变为1002（自动加1），再根据指令使PC变为1003，跳转到指令3执行

5.3.2 操作数寻址方式

操作数来源通常有3种：

1. 操作数直接来自指令地址字段： `MOV AL, 23H`
2. 操作数存放在寄存器中： `MOV AL, BL`
3. 操作数存放在存储器中： `MOV AL, [2300H]`

常用操作数寻址方式：立即寻址、直接寻址、间接寻址、寄存器寻址、寄存器间接寻址、相对寻址、变址寻址、基址寻址、堆栈寻址等。

指令的地址码A字段可细分为**寻址方式I字段**和**形式地址D字段**。寻址方式I字段也称寻址方式特征码。

设最终操作数为S，有效地址为EA，则 **$S = (EA)$** ，()表示访问地址为EA的主存或寄存器的内容。

1) 立即寻址

示例：MOV EAX, 2008H

- 形式地址D就是操作数S本身： **$S = D = 2008H$**
- 执行后：EAX = 2008H
- **指令执行阶段不需要访问主存**（数据不在主存，但指令在主存）

2) 直接寻址

示例：MOV EAX, [2008H]

- 有效地址由形式地址D直接给出： **$EA = D = 2008H$** ， $S = [2008H]$
- 假设主存2008H的内容为A0A0H，执行后：EAX = A0A0H
- 地址直观，执行阶段需访问**一次主存**
- 寻址范围受限于地址字段D的宽度

3) 寄存器寻址

示例：MOV EAX, ECX

- 形式地址D表示寄存器编号， **$EA = D$** ， **$S = R[D]$**
- **所有寻址方式中最快，执行阶段不需要访问主存**

4) 间接寻址

示例：MOV EAX, @2008H

- 形式地址D给出的是操作数的间接地址： **$EA = (D)$** ， $S = (EA)$
- 例5.2：主存2008H内容为A0A0F000H，主存A0A0F000H内容为5000H，则 $S = 5000H$
- 执行阶段需访问**至少两次主存**（取地址+取操作数，多次间址则多次访存）

5) 寄存器间接寻址

示例：MOV AL, [EBX]

- 形式地址D给出寄存器编号，有效地址为寄存器内容： $EA = R[D]$ ， $S = (EA)$
- 假设EBX=2010H，主存2010H内容为60H，则S=60H，执行后AL=60H
- 执行阶段只需访问**一次主存**

6) 相对寻址

示例：MOV EAX, 32[PC]

- 形式地址D为偏移量，有效地址为PC内容加偏移量： $EA = PC + D$
- 注意：PC的内容为**取指令完成后**PC的值（需 $PC+n \rightarrow PC$ ）
- 常用于跳转指令，如 JMP LOOP 中的LOOP就是相对寻址

例题5.3：某计算机指令字长为定长16位，内存按字节寻址，数据采用补码表示，PC值在取指令阶段完成修改。

(1) 若相对寻址指令的当前地址为2003H，要求数据有效地址为200AH，形式地址字段的值是多少？

解： $PC = 2003H + 2H = 2005H$ （16位=2字节）

$D = EA - PC = 200AH - 2005H = 5H$

(2) 若转移指令当前地址为2008H，要求转移后目标地址为2001H，形式地址字段的值是多少？

解： $PC = 2008H + 2H = 200AH$

$D = EA - PC = 2001H - 200AH = -9 = F7H$

7) 变址寻址

示例：MOV EAX, 32[ESI]

- 有效地址为变址寄存器X的内容加上形式地址D： $EA = R[X] + D$
- **D通常不变，X的内容可变**
- 用于**数组寻址**：D存放数组首地址，X存放元素偏移量（如A[4]，A为首地址，4为偏移量）
- **面向用户**，主要用于解决程序循环问题

8) 基址寻址

示例：MOV EAX, 32[EBX]

- 有效地址为基址寄存器B的内容加上形式地址D： $EA = R[B] + D$
- **D通常可变，B的内容不变**
- **面向系统**，主要用于程序的重定位（如页表基址寄存器PTBR）
- 多道程序环境下，用户按相对地址编写，操作系统给基址寄存器赋值，将虚地址转化为实地址

相对寻址、变址寻址、基址寻址三种非常类似，也称**偏移寻址**。

9) 堆栈寻址

堆栈（Stack）以**先进后出（FILO）**方式存储数据。有**存储器堆栈**和**寄存器堆栈**两种。

（1）存储器堆栈

SP为堆栈指针寄存器（Stack Pointer）。

- **入栈**： $SP = SP - 1$ ， $M[SP] = R$ （假设8位数据）
- **出栈**： $R = M[SP]$ ， $SP = SP + 1$

注意：16位数据用 $SP \pm 2$ ，32位数据用 $SP \pm 4$ 。

（2）寄存器堆栈

不需要堆栈指示器SP。

特性	寄存器堆栈	存储器堆栈
栈顶	固定不动	随操作移动
数据	堆栈操作时数据移动	数据不动
速度	快	慢
容量	有限	很大
控制	必须专用堆栈指令	不一定

10) 其他寻址方式

- （1）**变址+间接寻址**：先变址再间接， $EA = (R[X] + D)$
- （2）**间接+变址寻址**：先间接再变址， $EA = R[X] + (D)$
- （3）**相对+间接寻址**：先相对再间接， $EA = (PC + D)$

5.4 指令类型

一个完善的指令系统应包括以下指令类型：

5.4.1 算术逻辑运算指令

包括定点/浮点数的加、减、乘、除等算术运算，与、或、非、异或等逻辑运算。有些计算机还设置十进制运算指令。

示例： ADD AL, BL 、 SUB AL, BL 、 AND AL, BL 、 XOR AL, BL

5.4.2 移位操作指令

算术移位：针对带符号数（算术左移、算术右移）

逻辑移位：针对无符号数（逻辑左移、逻辑右移）

逻辑左移和算术左移是一样的。

循环移位：

- 小循环：循环左移、循环右移
- 大循环：带进位循环左移、带进位循环右移（涉及CF标志位）

5.4.3 数据传送指令

主要完成两个部件之间的数据传送。

指令类型	说明	示例
通用数据传送（MOV）	寄存器↔寄存器、寄存器↔存储器、存储器↔存储器	MOV AX, BX
LOAD/STORE	用于访问存储器	LW , SW
交换指令（SWAP）	双向数据传送	XCHG
串传送指令（MOVS）	加重复前缀REP可实现数据块传送	REP MOVS
堆栈指令（PUSH/POP）	压栈/出栈	PUSH AX

5.4.4 堆栈操作指令

- **PUSH：**将寄存器内容压入堆栈
- **POP：**将堆栈内容弹出到寄存器

5.4.5 字符串处理指令

非数值处理指令，包括字符传送、字符串比较、查找、抽取、转换等。不是所有计算机都支持。

5.4.6 程序控制指令

(1) 转移指令

- 无条件转移： `JMP LOOP`
- 条件转移： `JZ LOOP` （结果为零则转移）

(2) 循环控制指令

示例： `LOOP L1` ， 每执行一次ECX减1， 判断ECX是否为0； 不为0则继续循环。

(3) 子程序调用和返回指令

- **CALL**： 子程序调用指令（转子指令）， 需保护断点（通常将断点压入堆栈）
- **RET**： 子程序返回指令， 从堆栈取出断点

CALL与转移指令的区别：

特性	转移指令	CALL指令
转移位置	同一程序内	不同程序之间
返回	不需要返回	需要返回原处
断点保护	不需要	需要
条件性	可有条件	通常无条件

5.4.7 输入输出指令

- **IN**： 从输入设备读入， `IN AL, DX`
- **OUT**： 向输出设备写出， `OUT DX, AL`

当外部设备与主存采用**统一编址**时， 不需要专用IN/OUT指令， 可使用访存指令实现。

5.4.8 其他指令

停机、 等待、 空操作、 开中断、 关中断、 自陷、 置条件码、 **特权指令**等。

特权指令一般只能用于操作系统或其他系统软件， 不直接提供给用户使用。

5.5 指令格式设计

指令系统是程序员所能看到的计算机的主要属性，在很大程度上决定了计算机系统的基本功能和程序性能。

5.5.1 设计考虑因素

- 满足完备性、规整性、有效性、兼容性和可扩展性
- 支持哪些指令类型（5.4节）
- 支持哪些数据类型和寻址方式（5.3.2节）
- 合理的指令格式（5.2节）
- 方便程序员编程，有利于编译系统设计，简化硬件实现，节省存储空间

5.5.2 指令编码格式的设计

(1) 定长指令格式

指令长度固定。如MIPS32，所有指令机器码都是32位。

- 优点：简化硬件，尤其简化指令译码部件设计
- 缺点：平均长度长，容易出现冗余码点，不利于扩展

(2) 变长指令格式

指令长度可变。如Intel x86，最短1字节，最长15字节。

- 优点：平均长度短，码点冗余少，易于扩展
- 缺点：格式不规整，不同指令取指时间不同，控制方式复杂

(3) 混合编码指令格式

定长和变长的综合。提供若干长度固定的指令字，既能减少目标代码长度，又能降低译码复杂度。

5.5.3 操作码的设计

满足完备性是基本要求。分定长结构和变长结构。

5.5.4 地址码的设计

地址码数量：三地址、双地址、单地址、零地址。目的是利用有限的位宽提供更大的寻址范围。

5.5.5 寻址方式的设计

两种方法：

- 1. 寻址方式与操作码一起编码
- 2. 设置专门的寻址方式字段（图5.5）

例题5.4：某计算机字长为16位，内存为64KB，指令采用单字长、单地址结构，要求至少支持80条指令和直接、间接、相对、变址4种寻址方式。请设计指令格式并计算每种寻址方式的访存空间范围。

解：

字段	位数	计算依据
操作码OP	7位	$2^7=128 > 80$
寻址方式I	2位	$2^2=4$ 种
形式地址D	7位	$16-7-2=7$

指令格式：OP(7位) + I(2位) + D(7位)

寻址方式	访存空间范围	说明
直接寻址	0 ~ 127	EA = D = 7位
间接寻址	0 ~ 65535	EA = (D) = 16位（存储器宽度=字长=16位）
相对寻址	0 ~ 65535	EA = PC + D = 16位
变址寻址	0 ~ 65535	EA = R[X] + D = 16位

补充例题

例题（唐朔飞）：某机字长16位，存储器直接寻址空间128字，变址时位移量-64~+63，16个通用寄存器均可作为变址寄存器。设计指令系统格式，满足：直接寻址的二地址指令3条、变址寻址的一地址指令6条、寄存器寻址的二地址指令8条、直接寻址的一地址指令12条、零地址指令32条。还有多少种代码未用？若安排寄存器寻址的一地址指令，还能容纳多少条？

解：

指令类型	OP编码	格式
直接寻址二地址	00~10（3条）	OP(2) A1(7) A2(7)

指令类型	OP编码	格式
变址寻址一地址	11 000~11 101 （6条）	OP(6) Rx(4) A(6)
寄存器寻址二地址	11110 000~11110 111 （8条）	OP(9) Ri(4) Rj(4)
直接寻址一地址	11111 0000~11111 1011 （12条）	OP(10) A(6)
零地址指令	11111 1100 0000000~11111 1100 0011111 （32条）	OP(16)

剩余代码：2⁹ - 32 = **480种**

若安排寄存器寻址的一地址指令（末4位为寄存器地址），还可容纳 **30条**。

5.6 CISC和RISC

5.6.1 复杂指令系统计算机（CISC）

CISC（Complex Instruction Set Computer）：计算机系统设计者在设计指令系统时，增加了越来越多的功能强大的复杂指令和更多的寻址方式。

发展动因：

- 更好支持高级语言，增加语义接近高级语言的指令
- 简化编译
- 满足系列计算机软件向后兼容需求（新计算机指令只能增加不能减少）
- 对操作系统的支持（多媒体指令、3D指令等）
- 在有限指令长度内基于扩展法实现更多指令，需压缩地址码长度，设计多种寻址方式

CISC特点：

特性	说明
指令系统	复杂庞大，指令数目多达200~300条
寻址方式	多
指令格式	多
指令字长	不固定
访存指令	不加限制

特性	说明
使用频率	各种指令相差大
执行时间	各种指令相差大
控制器	大多数采用微程序控制器

5.6.2 精简指令系统计算机（RISC）

RISC（Reduced Instruction Set Computer）：针对CISC指令种类太多、格式不规范、寻址方式太多的缺点，通过减少指令种类、规范指令格式和简化寻址方式，方便处理器内部并行处理，提高性能。

"80-20"规律：80%的程序只用到了20%的指令。

RISC特点：

特性	说明
指令选择	优先选择使用频率最高的简单指令和很有用但不复杂的指令
执行周期	大多数指令在一个时钟周期内完成
访存结构	采用LOAD/STORE结构，只有LOAD/STORE指令可访问主存，其余指令只能对寄存器操作数处理
指令格式	简单、固定，指令长度固定，有利于提高流水线效率，常采用三地址格式
寄存器结构	面向寄存器，CPU内设大量通用寄存器
控制逻辑	采用硬布线控制逻辑，不用或少用微程序控制
编译优化	注重编译优化，有效支持高级语言程序

常见RISC处理器：X86（实际为CISC）、ARM、RISC-V、MIPS

5.7 指令系统举例

5.7.1 PDP-11指令系统

PDP-11是DEC公司1970年推出的经典16位小型计算机，IBM PC出现之前最流行的小型机。

指令格式：

- 典型的16位CISC指令系统
- 单字长（16位），有单操作数和双操作数两种格式
- 双操作数时可通过变址寻址增加1~2个立即数index字段，扩展为双字长（32位）、三字长（48位）
- 采用扩展操作码技术

寻址特征：

- 寻址特征字段I为3位，有8种寻址方式
- 寄存器字段A为3位，可寻址8个寄存器R0_{R7}（R0R5通用，R6=SP，R7=PC）

表5.1 PDP-11寻址方式：

模式	寻址方式	汇编语法	有效地址/操作数	说明
0	寄存器寻址	Ri	S=(Ri)	寄存器值就是操作数
1	寄存器间接寻址	(Ri)	EA=(Ri)	寄存器的值是操作数地址
2	自增寻址	(Ri)+	EA=(Ri), Ri++	寄存器值是地址，取数后自增
3	自增间接寻址	@(Ri)+	EA=((Ri)), Ri++	寄存器值是地址的地址
4	自减寻址	-(Ri)	EA=(Ri), Ri--	先自减，结果是操作数地址
5	自减间接寻址	@-(Ri)	EA=((Ri)), Ri--	先自减，结果是地址的地址
6	变址寻址	index(Ri)	EA=(Ri)+index	寄存器值+16位index
7	变址间址寻址	@index(Ri)	EA=((Ri))+index	寄存器值+index，取间接地址

当寄存器字段为R7（PC）时，可扩展出4种寻址方式（表5.2）：

模式	寻址方式	汇编语法	有效地址	功能
2	立即数寻址	#n	S=index	操作数是指令后续一个字
3	直接寻址	@ #n	EA=index	操作数地址是指令后续一个字
6	相对寻址	A/A(PC)	EA=PC+index	操作数地址=PC+偏移量
7	相对间接	@A/@A(PC)	EA=(PC+index)	操作数地址的地址=PC+偏移量

PDP-11的自增、自减寻址方式方便堆栈操作（SP--），也影响了C语言语法（i++、++i、i--、--i、(--i)=(j++)）。

5.7.2 Intel X86指令系统

由16位8086/8088发展而来的32位CISC指令系统。

80386 CPU寄存器：

- 32位通用寄存器：EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI
- 32位程序计数器：EIP
- 32位状态寄存器：EFLAGS
- 16位段寄存器：CS、DS、SS、ES、FS、GS

1. 指令格式

采用变长指令格式（图5.15），包括：前缀Prefix（可选）、操作码OP（必需）、Mod R/M（可选）、SIB（可选）、偏移量Disp（可选）、立即数Imm（可选）。最短1字节，最长15字节。

2. 前缀Prefix

包括4部分：

前缀类型	说明	示例
指令前缀 (1字节)	锁定前缀（Lock）用于多处理器共享存储器排他性访问； 重复前缀（REPNE/REPZ等）用于字符串循环处理	
段前缀 (1字节)	指定使用哪个段寄存器取代默认段寄存器	2EH=CS, 36H=SS, 3EH=DS, 64H=FS, 65H=GS
操作数大小 (1字节)	在32位和16位操作数之间切换	字段值66H
地址大小 (1字节)	在32位和16位地址空间之间切换	字段值67H

3. 操作码OP

格式	OP位数	说明
单地址	5位	Reg=3位表示8个通用寄存器，如push、pop、inc、dec
双地址	6位	d位表示目的/源，w位表示操作数宽度

格式	OP位数	说明
扩展	8位	零/单/双地址指令
双字节	16位	高8位=0FH，低8位为扩展操作码

4. Mod R/M字段

用于描述操作数及其寻址方式。包括：

- **Mod（2位）**：与R/M一起构成8种寻址方式
- **Reg/OP（3位）**：寄存器编号或操作码扩展
- **R/M（3位）**：另一个操作数（寄存器或存储器）

表5.4 Mod R/M对应的寻址方式：

Mod	R/M值	操作数类型	寻址方式	有效地址
00	-	存储器	寄存器间接寻址	EA=R[R/M]
00	100	存储器	基址+比例变址	EA=SIB
00	101	存储器	直接寻址	EA=Disp32
01	-	存储器	寄存器相对寻址	EA=R[R/M]+Disp8
01	100	存储器	基址+比例变址+偏移量	EA=SIB+Disp8
10	-	存储器	寄存器相对寻址	EA=R[R/M]+Disp32
10	100	存储器	基址+比例变址+偏移量	EA=SIB+Disp32
11	-	寄存器	寄存器寻址	S=R[R/M]

5. SIB字段

Scale（2位）+ Index（3位）+ Base（3位）

有效地址：**EA = R[Base] + 2^Scale × R[Index]**

比例因子：Scale=00/01/10/11，对应1/2/4/8。

6. 偏移量Disp和立即数Imm

- Disp：8位/16位/32位/无，取决于Mod R/M中的寻址方式
- Imm：8位/16位/32位/无，取决于操作码

7. 指令译码流程

- 1. 前缀分析：判断有无前缀，最多4字节
- 2. 操作码译码：判断单字节还是双字节操作码
- 3. 寻址方式译码：根据Mod R/M确定操作数和寻址方式，配合SIB、Disp、Imm计算地址

8. x86寻址方式

采用段页式存储管理，3类地址：逻辑地址→线性地址→物理地址。

- 实模式：物理地址(20位) = 段寄存器(16位)左移4位 + 段内偏移地址
- 保护模式：线性地址(32位) = 段基址(32位) + 段内偏移地址，再通过MMU转换为物理地址

表5.6 x86主要寻址方式：

序号	寻址方式	有效地址/操作数	示例
1	立即数寻址	S=Disp	MOV EAX, 1000
2	直接寻址	EA=Disp	MOV EAX, [1080H]
3	寄存器寻址	S=R[R/M]	MOV EAX, ECX
4	寄存器间接寻址	EA=R[R/M]	MOV EAX, [EBX]
5	基址寻址	EA=R[R/M]+Disp8	MOV EAX, [ESI+100H]
6	基址+比例变址	EA=S×index+Base	MOV EAX, [EBX+EDI×4]
7	基址+比例变址+偏移量	EA=S×index+Base+Disp	MOV EAX, [EBX+EDI×4+66]
8	相对寻址	EA=PC+Disp	JMP 1000H

例题5.5：32位运行环境下，给出指令字对应的汇编代码。

指令字	汇编代码	分析要点
01C2H	ADD EDX, EAX	OP=01, d=0, w=1, Mod=11寄存器寻址, Reg=EAX, R/M=EDX
2E 033BH	ADD EDI, CS:[EBX]	Prefix=2E(CS段), OP=03, d=1, Mod=00, R/M=011(EBX)
034C BB66H	ADD ECX, [EBX+EDI×4+66]	OP=03, Mod=01, SIB: Base=EBX, Index=EDI, Scale=4, Disp8=66

指令字	汇编代码	分析要点
8304 BB66H	ADD [EBX+EDI×4], 66	OP=83, Mod=00, SIB: Base=EBX, Index=EDI, Scale=4, Imm8=66

5.7.3 MIPS指令系统

MIPS：Microprocessor without Interlocked Pipeline Stages（无互锁流水线的微处理器）。1981年斯坦福大学Hennessy教授研究小组研制。包括MIPS32（32位）和MIPS64（64位）。**龙芯处理器**基于MIPS指令系统。

1. 寄存器

- 32个32位通用寄存器：0 31（*zero*, *at*, ..., *\$ra*）
- 32个32位单精度浮点寄存器：f0~f31（可配对为16个64位浮点寄存器）
- 2个特殊寄存器：*hi*、*lo*（保存乘除运算结果）

2. 指令格式

所有指令为32位定长，包括R型、I型、J型三种。

- OP=6位；R型OP=000000，功能由funct字段（6位）决定
- Rs、Rt、Rd均为5位，可访问32个通用寄存器
- shamt为5位移位变量
- 立即数有16位（I型）和26位（J型）两种
- **没有独立的寻址方式字段**，寻址方式由操作码决定

（1）R型指令（寄存器型）

操作数只能是寄存器，结果也存入寄存器（RR型）。OP=000000，功能由funct指定。

funct	指令	功能	备注
00	sll rd, rt, shamt	R[rd]=R[rt]<<shamt	逻辑左移
02	srl rd, rt, shamt	R[rd]=R[rt]>>shamt	逻辑右移
03	sra rd, rt, shamt	R[rd]=R[rt]>>shamt	算术右移
08	jr rs	PC=R[rs]	寄存器跳转

funct	指令	功能	备注
09	jalr rs	$R[31]=PC+8, PC=R[rs]$	子程序调用
32	add rd, rs, rt	$R[rd]=R[rs]+R[rt]$	加法（溢出异常）
34	sub rd, rs, rt	$R[rd]=R[rs]-R[rt]$	减法（溢出异常）
36	and rd, rs, rt	$R[rd]=R[rs]\&R[rt]$	逻辑与
37	or rd, rs, rt	$R[rd]=R[rs]\ R[rt]$	逻辑或
42	slt rd, rs, rt	$R[rd]=(R[rs]<R[rt])?1:0$	小于置位

例题5.6：写出 add \$s1, \$t0, \$s4 各字段的十进制值。

解：add为R型指令，OP=0，funct=32。rd=s1 =17，rs=t0 =8，rt=s4 =20，shamt=0。

OP	rs	rt	rd	shamt	funct
0	8	20	17	0	32

(2) I型指令（立即数型）

OP	指令	功能	备注
04	beq rs, rt, imm	if($R[rs]==R[rt]$) $PC=PC+4+imm<<2$	相等跳转， imm左移2位确保4字节对齐
05	bne rs, rt, imm	if($R[rs]!=R[rt]$) $PC=PC+4+imm<<2$	不等跳转
08	addi rt, rs, imm	$R[rt]=R[rs]+imm$	立即数加
35	lw rt, imm(rs)	$R[rt]=M[R[rs]+imm]$	取字
43	sw rt, imm(rs)	$M[R[rs]+imm]=R[rt]$	存字

(3) J型指令（无条件转移型）

26位直接地址，伪直接寻址。目标地址由PC+4的高4位与address左移2位后拼接得到。

OP	指令	功能
02	j address	$PC \leftarrow \{(PC+4)_{31:28}, \text{address}, 00\}$
03	jal address	$R[31] \leftarrow PC+8, PC \leftarrow \{(PC+4)_{31:28}, \text{address}, 00\}$

3. MIPS寻址方式

只有5种寻址方式：

序号	寻址方式	有效地址/操作数	示例
1	立即数寻址	$S = \text{imm}$	addi rt, rs, imm
2	寄存器寻址	$S = R[\text{rt}]$	add rd, rs, rt
3	基址寻址	$EA = R[\text{rs}] + \text{imm}$	lw rt, imm(rs)
4	相对寻址	$EA = PC + 4 + \text{Disp}$	beq rs, rt, imm
5	伪直接寻址	$EA = \{(PC+4)_{31:28}, \text{address}, 00\}$	j address

5.7.4 RISC-V指令系统

RISC-V（发音"RISC Five"）是第五代RISC指令系统，结合ARM、MIPS等RISC优势，完全从零重新设计的**开源指令系统**。

采用模块化设计：

- 32位RISC-V指令集（RV32G）包括核心指令集RV32I（47条指令）和4个标准扩展集：RV32M（乘除法）、RV32F（单精度浮点）、RV32D（双精度浮点）、RV32A（原子操作）
- 阿里巴巴**平头哥玄铁处理器**采用RISC-V架构

1. RISC-V通用寄存器

32个32位通用寄存器x0~x31：

编号	助记符	功能
x0	zero	恒零值
x1	ra	返回地址

编号	助记符	功能
x2	sp	栈指针
x3	gp	全局指针
x4	tp	线程寄存器
x5~x7	t0~t2	临时变量（调用者保存）
x8	s0/fp	被调用者保存/帧指针
x10~x11	a0~a1	子程序参数/返回值
x12~x17	a2~a7	子程序参数
x18~x27	s2~s11	被调用者保存寄存器
x28~x31	t3~t6	临时变量

2. RISC-V指令格式

32位定长，OP=7位，有6种指令格式。没有寻址方式字段，由操作码决定。**各字段位置固定**，提高译码速度。

（1）R型指令（寄存器型）

OP=33H，功能由funct3（3位）+ funct7（7位）决定。

类型	示例	功能	同类指令
算术逻辑	add rd, rs1, rs2	R[rd]=R[rs1]+R[rs2]	add、sub、xor、or、and
关系运算	slt rd, rs1, rs2	R[rd]=(R[rs1]<R[rs2])?1:0	slt、sltu
移位	sll rd, rs1, rs2	R[rd]=R[rs1]<<R[rs2]	sll、srl、sra

（2）I型指令（立即数型）

类型	示例	功能	同类指令
算术逻辑	addi rd, rs1, imm	R[rd]=R[rs1]+imm	addi、xori、ori、andi
关系运算	slti rd, rs1, imm	R[rd]=R[rs1]<imm	slti、sltiu
移位	slli rd, rs1, imm	R[rd]=R[rs1]<<imm	slli、srli、srai

类型	示例	功能	同类指令
访存	lw rd, imm(rs1)	$R[rd] = M[R[rs1] + imm]$	lb、lbu、lh、lhu、lw
系统控制	jalr rd, rs1, imm	$PC = R[rs1] + imm, R[rd] = PC + 4$	jalr

(3) S型指令（写存储器型）

类型	示例	功能
访存	sw rs2, imm(rs1)	$M[R[rs1] + imm] = R[rs2]$

(4) B型指令（条件分支型）

类型	示例	功能
分支	beq rs1, rs2, imm	$if(R[rs1] == R[rs2]) PC = PC + imm \ll 1$

同类：beq、bne、blt、bge、bltu、bgeu

(5) U型指令（更大立即数型）

类型	示例	功能
立即数加载	lui rd, imm	$R[rd] = imm \ll 12$
PC相关加载	auipc rd, imm	$R[rd] = PC + imm \ll 12$

(6) J型指令（无条件跳转型）

类型	示例	功能
子程序调用	jal rd, imm	$PC = PC + imm \ll 1, R[rd] = PC + 4$

3. RISC-V核心指令集RV32I（47条）

类别	数量	指令
移位指令	6条	sll、slli、srl、srli、sra、srai
算术运算	5条	add、addi、sub、lui、auipc
逻辑运算	6条	xor、xori、or、ori、and、andi

类别	数量	指令
比较指令	4条	slt、slti、sltu、sltiu
分支指令	6条	beq、bne、blt、bge、bltu、bgeu
跳转指令	2条	jal、jalr
同步指令	2条	fence、fence.i
环境指令	2条	ecall、ebreak
CSR指令	6条	csrrw、csrrs、csrrc、csrrwi、csrrsi、csrrci
取数指令	5条	lb、lh、lbu、lhu、lw
存数指令	3条	sb、sh、sw

4. RISC-V寻址方式

只有4种（比MIPS少伪直接寻址）：

序号	寻址方式	有效地址/操作数	示例
1	立即数寻址	S=imm	addi rd, rs1, imm
2	寄存器寻址	S=R[rs1]	add rd, rs1, rs2
3	基址寻址	EA=R[rs1]+imm	lw rd, imm(rs1)
4	相对寻址	EA=PC+imm<<1	beq rs1, rs2, imm

5.7.5 ARMv7指令系统（补充）

ARM：Advanced RISC Machines。1991年成立于英国剑桥，是32位嵌入式RISC微处理器领先IP核供应商。ARM不制造芯片，通过转让设计方案由合作伙伴生产。

ARMv7三个家族：

家族	名称	面向
Cortex-A	高性能处理器	移动计算（智能手机、平板、服务器），支持MMU，面向虚拟内存操作系统

家族	名称	面向
Cortex-R	实时处理器	要求高可靠性、容错、实时响应的嵌入式系统
Cortex-M	低成本低功耗	MCU和终端应用（智能测量、人机接口、汽车控制、医疗器械）

ARM指令集：

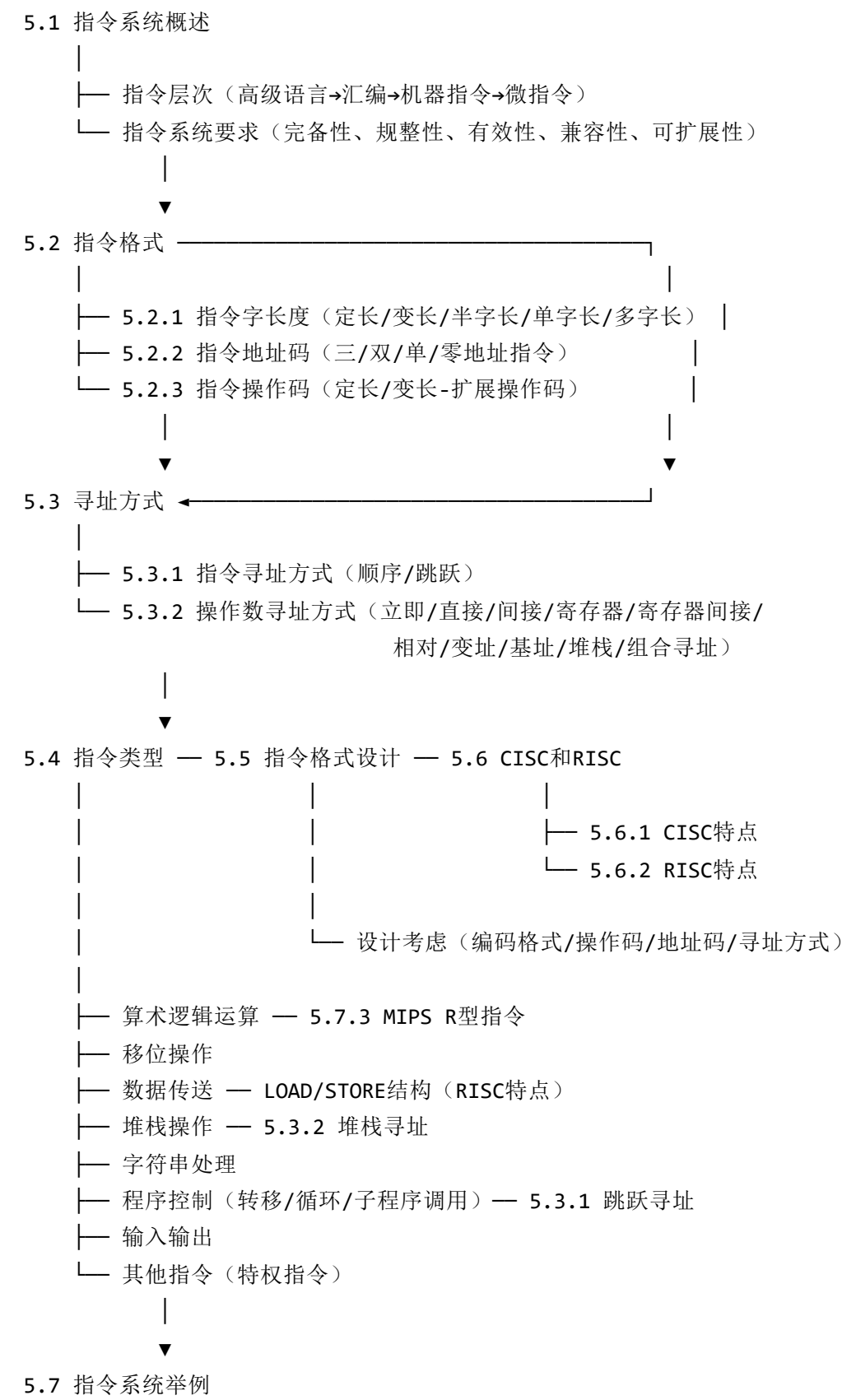
- ARM指令集：32位
- Thumb指令集：16位、32位（ARM的子集）
- ThumbEE指令集：Thumb的变体

ARMv7寄存器：

- 未备份寄存器：R0~R7
- 备份寄存器：R8~R14（R13=SP堆栈指针，R14=LR链接寄存器）
- 程序计数器PC：R15
- CPSR：当前程序状态寄存器
- SPSR：备份程序状态寄存器

知识点关联总览

各节知识点关联图



- |
- └ 5.7.1 PDP-11（经典CISC，16位，自增/自减寻址影响C语言语法）
- └ 5.7.2 Intel X86（变长CISC，前缀/Mod R/M/SIB复杂译码）
- └ 5.7.3 MIPS（经典RISC，32位定长，R/I/J三种格式）
- └ 5.7.4 RISC-V（开源RISC，6种指令格式，47条核心指令）
- └ 5.7.5 ARMv7（嵌入式RISC，三种家族/Thumb指令集）

CISC vs RISC vs 实际处理器对比

特性	CISC代表（X86）	RISC代表（MIPS/RISC-V/ARM）
指令字长	变长（1~15字节）	定长（32位）
寻址方式	多种（8种以上）	少（4~5种）
访存指令	任意指令可访存	仅LOAD/STORE
寄存器数量	较少	较多（32个）
控制器	微程序控制	硬布线控制
执行周期	不固定	大多单周期
指令格式	复杂（前缀+Op+ModR/M+SIB）	简单规整（字段位置固定）
设计目标	减少指令条数	提高流水线效率

关键关联

关联点	说明
5.2.3 扩展操作码 → 5.5 指令格式设计	扩展操作码是变长操作码的核心技术
5.3 寻址方式 → 5.7.2 X86 Mod R/M	Mod R/M字段本质是寻址方式的编码
5.3.2 堆栈寻址 → 5.4.6 子程序调用	CALL/RET依赖堆栈保护/恢复断点
5.4.3 数据传送 → 5.6.2 RISC LOAD/STORE	RISC只允许LOAD/STORE访存
5.6 CISC/RISC → 5.7 所有举例	PDP-11/X86是CISC代表，MIPS/RISC-V/ARM是RISC代表
5.7.1 PDP-11自增寻址 → C语言i++语法	C语言的++/--运算符源自PDP-11

关联点	说明
5.7.3 MIPS → 龙芯处理器	龙芯基于MIPS指令系统
5.7.4 RISC-V → 平头哥玄铁	玄铁处理器采用RISC-V架构

文档整理时间：2026年5月
课程：计算机组成原理 第5章 指令系统
教材参考：厦门大学信息学院软件工程系 张海英